Initia MiniEVM 아키텍처 심층 분석

목차

1. MiniEVM 개요
2. 핵심 아키텍처
3. x/evm 모듈 분석
4. Keeper 구조 분석
5. EVM 실행 플로우
6. ERC20/ERC721 통합
7. Cosmos ↔ EVM 브릿지
8. JSON-RPC 서버
9. OPinit 통합
10. Octane vs MiniEVM 비교

---

1. MiniEVM 개요

1.1 MiniEVM이란?

MiniEVM은 Initia Labs가 개발한 **Optimistic Rollup L2 체인**으로, EVM을 Cosmos SDK 애플리케이션으로 통합한 구현입니다.

> 출처: `minievm/README.md` (실제 레포지토리)

```
MiniEVM is an optimistic rollup consumer chain powered by EVM, designed to
simplify the process of bootstrapping an L2 network. The main advantage of
using MiniEVM is that the users can leverage the OPinit stack for enhanced
security and utilize all the Initia ecosystem tooling from day one.
```

1.2 주요 특징

특징	설명
Optimistic Rollup	L2 확장성 솔루션
EVM 통합	완전한 EVM 호환성
Cosmos SDK 기반	Cosmos 생태계 도구 활용
OPinit Stack	Initia의 보안 스택
No Validator Setup	별도 validator 그룹 불필요
Built-in Indexer	EVM 이벤트 인덱싱

1.3 아키텍처 다이어그램

```
┌──────────────────────────────────────────────────────────┐
│                  Initia L1 (DA Layer)                    │
│  - Data Availability & Consensus                         │
│  - OPinit Bridge                                         │
└──────────────────────┬───────────────────────────────────┘
                       │ OPinit Protocol
                       ▼
┌──────────────────────────────────────────────────────────┐
│              MiniEVM L2 (Cosmos SDK App)                 │
│                                                          │
│  ┌──────────────────────────────────────────┐           │
│  │  x/evm Module (EVM 통합)                  │           │
│  │  ┌──────────────────────────────────────┐ │           │
│  │  │  Keeper                              │ │           │
│  │  │  - EVM state management              │ │           │
│  │  │  - Contract deployment               │ │           │
│  │  │  - ERC20/ERC721 bridges              │ │           │
│  │  └──────────────────────────────────────┘ │           │
│  │  ┌──────────────────────────────────────┐ │           │
│  │  │  EVM Core (go-ethereum fork)         │ │           │
│  │  │  - State transition                  │ │           │
│  │  │  - Opcode execution                  │ │           │
│  │  └──────────────────────────────────────┘ │           │
│  └──────────────────────────────────────────┘           │
│                                                          │
│  ┌──────────────────────────────────────────┐           │
│  │  Other Cosmos Modules                    │           │
│  │  - x/bank, x/auth, x/staking, etc.       │           │
│  └──────────────────────────────────────────┘           │
│                                                          │
│  ┌──────────────────────────────────────────┐           │
│  │  JSON-RPC Server                         │           │
│  │  - eth_*, net_*, web3_* endpoints        │           │
│  │  - WebSocket support                     │           │
│  └──────────────────────────────────────────┘           │
│                                                          │
│  ┌──────────────────────────────────────────┐           │
│  │  Indexer                                 │           │
│  │  - Block/Tx indexing                     │           │
│  │  - Event indexing                        │           │
│  └──────────────────────────────────────────┘           │
└──────────────────────────────────────────────────────────┘
```

---

2. 핵심 아키텍처

2.1 디렉터리 구조

실제 레포지토리 구조 (출처: `minievm/` root)

```
minievm/
├── x/evm/                    ← EVM 모듈 (핵심)
│   ├── keeper/             ← State management
│   ├── types/              ← Proto definitions
│   ├── contracts/          ← Predeployed contracts
│   ├── client/             ← CLI commands
│   └── abci.go             ← ABCI integration
├── jsonrpc/                ← JSON-RPC server
│   ├── server/             ← HTTP/WebSocket server
│   ├── backend/            ← Backend implementation
│   └── namespaces/         ← eth_*, net_*, web3_*
├── indexer/                ← Block/Tx/Event indexer
│   ├── db/                 ← Database layer
│   └── types/              ← Indexer types
├── app/                    ← Cosmos SDK app
├── cmd/                    ← CLI binaries
└── tests/                  ← Integration tests
```

2.2 기술 스택

```go
// go.mod (주요 의존성)
require (
    github.com/cosmos/cosmos-sdk v0.50.x
    github.com/ethereum/go-ethereum v1.14.x  // EVM core
    github.com/cometbft/cometbft v0.38.x
    github.com/initia-labs/initiavm v0.x     // Move VM (optional)
    cosmossdk.io/collections                  // Collections framework
)
```

---

3. x/evm 모듈 분석

3.1 PreBlock Hook

실제 구현 (출처: `x/evm/abci.go` lines 13-23)

```go
// PreBlock track latest 256 block hashes
func PreBlock(ctx sdk.Context, k *keeper.Keeper) (sdk.ResponsePreBlock, error) {
    defer telemetry.ModuleMeasureSince(types.ModuleName, time.Now(), telemetry.MetricKeyEndBlocker)

    height := uint64(ctx.BlockHeight())
    if height <= 1 {
        return sdk.ResponsePreBlock{}, nil
    }

    // current block header hash is created by the previous block,
    // so we track it with height-1
    return sdk.ResponsePreBlock{}, k.TrackBlockHash(ctx, height-1, common.BytesToHash(ctx.HeaderHash()))
}
```

역할:

- EVM의 `BLOCKHASH` opcode 지원
- 최근 256개 블록 해시 추적
- Cosmos SDK의 PreBlock hook 활용 (v0.50+)

3.2 모듈 파일 구성

```
x/evm/
├── abci.go              ← PreBlock implementation
├── autocli.go           ← AutoCLI configuration
├── module.go            ← Module definition
├── keeper/
│   ├── keeper.go        ← Keeper struct & constructor
│   ├── msg_server.go    ← Msg handlers (Create, Call, etc.)
│   ├── context.go       ← EVM context setup
│   ├── erc20.go         ← ERC20 bridge logic
│   ├── erc721.go        ← ERC721 bridge logic
│   └── ...
```

---

4. Keeper 구조 분석

4.1 Keeper 정의

실제 구조체 (출처: `x/evm/keeper/keeper.go` lines 22-71)

```go
type Keeper struct {
    ac           address.Codec
    cdc          codec.Codec
    storeService corestoretypes.KVStoreService

    // Cosmos SDK module keepers
    accountKeeper       types.AccountKeeper
    bankKeeper          types.BankKeeper
    communityPoolKeeper types.CommunityPoolKeeper
    ibcHookKeeper       types.IBCHookKeeper
    gasPriceKeeper      types.GasPriceKeeper

    // ERC bridges
    erc20Keeper         types.IERC20Keeper
    erc20StoresKeeper   types.IERC20StoresKeeper
    erc721Keeper        types.IERC721Keeper
    txUtils             types.TxUtils

    // gRPC routers
    msgRouter  baseapp.MessageRouter
    grpcRouter types.GRPCRouter

    config evmconfig.EVMConfig
    authority string  // x/gov module account

    // Collections (state storage)
    Schema  collections.Schema
    Params  collections.Item[types.Params]
    VMStore collections.Map[[]byte, []byte]  // EVM state

    // Predeployed contracts
    ERC20FactoryAddr  collections.Item[[]byte]
    ERC20WrapperAddr  collections.Item[[]byte]
    ConnectOracleAddr collections.Item[[]byte]

    // ERC20 bridges
    ERC20s                     collections.KeySet[[]byte]
    ERC20Stores                collections.KeySet[collections.Pair[[]byte, []byte]]
    ERC20DenomsByContractAddr collections.Map[[]byte, string]
    ERC20ContractAddrsByDenom collections.Map[string, []byte]

    // ERC721 bridges
    ERC721ClassURIs              collections.Map[[]byte, string]
    ERC721ClassIdsByContractAddr collections.Map[[]byte, string]
    ERC721ContractAddrsByClassId collections.Map[string, []byte]

    // EVM block hashes (for BLOCKHASH opcode)
    EVMBlockHashes collections.Map[uint64, []byte]

    queryCosmosWhitelist types.QueryCosmosWhitelist
    cosmosCallbackABI    *abi.ABI
}
```

핵심 필드 분석:

1. `VMStore` : EVM 상태 저장소 (account state, contract storage)
2. `EVMBlockHashes` : BLOCKHASH opcode를 위한 블록 해시 저장
3. `ERC20*/ERC721*` : Cosmos ↔ EVM 토큰 브릿지
4. `msgRouter` : Cosmos 메시지 라우팅 (EVM → Cosmos 호출)
5. `grpcRouter` : gRPC 쿼리 라우팅 (EVM → Cosmos 읽기)

4.2 Keeper 생성자

```go
// x/evm/keeper/keeper.go (lines 73-150)
func NewKeeper(
    ac address.Codec,
    cdc codec.Codec,
    storeService corestoretypes.KVStoreService,
    accountKeeper types.AccountKeeper,
    bankKeeper types.BankKeeper,
    communityPoolKeeper types.CommunityPoolKeeper,
    ibcHookKeeper types.IBCHookKeeper,
    gasPriceKeeper types.GasPriceKeeper,
    msgRouter baseapp.MessageRouter,
    grpcRouter *baseapp.GRPCQueryRouter,
    authority string,
    evmConfig evmconfig.EVMConfig,
    queryCosmosWhitelist types.QueryCosmosWhitelist,
) *Keeper {
    sb := collections.NewSchemaBuilder(storeService)

    // Default config
    if evmConfig.ContractSimulationGasLimit == 0 {
        evmConfig.ContractSimulationGasLimit = evmconfig.DefaultContractSimulationGasLimit
    }

    // Cosmos callback ABI
    cosmosCallbackABI, err := i_cosmos_callback.ICosmosCallbackMetaData.GetAbi()
    if err != nil {
        panic(err)
    }

    k := &Keeper{
        ac:                 ac,
        cdc:                cdc,
        storeService:       storeService,
        accountKeeper:      accountKeeper,
        bankKeeper:         bankKeeper,
        communityPoolKeeper: communityPoolKeeper,
        ibcHookKeeper:      ibcHookKeeper,
        gasPriceKeeper:     gasPriceKeeper,
        authority:          authority,
        msgRouter:          msgRouter,
        grpcRouter:         grpcRouter,
        config:             evmConfig,

        // Collections setup
        Params:  collections.NewItem(sb, types.ParamsKey, "params", codec.CollValue[types.Params](cdc)),
        VMStore: collections.NewMap(sb, types.VMStorePrefix, "vm_store", collections.BytesKey, collections.BytesValue),

        ERC20WrapperAddr:  collections.NewItem(sb, types.ERC20WrapperAddrKey, "erc20_wrapper_addr", collections.BytesValue),
        ERC20FactoryAddr:  collections.NewItem(sb, types.ERC20FactoryAddrKey, "erc20_factory_addr", collections.BytesValue),
        ConnectOracleAddr: collections.NewItem(sb, types.ConnectOracleAddrKey, "connect_oracle_addr", collections.BytesValue),

        ERC20s:                   collections.NewKeySet(sb, types.ERC20sPrefix, "erc20s", collections.BytesKey),
        ERC20Stores:              collections.NewKeySet(sb, types.ERC20StoresPrefix, "erc20_stores", collections.PairKeyCodec(collecti
        ERC20DenomsByContractAddr: collections.NewMap(sb, types.ERC20DenomsByContractAddrPrefix, "erc20_denoms_by_contract_addr", colle
        ERC20ContractAddrsByDenom: collections.NewMap(sb, types.ERC20ContractAddrsByDenomPrefix, "erc20_contract_addrs_by_denom", colle

        ERC721ClassURIs:              collections.NewMap(sb, types.ERC721ClassURIPrefix, "erc721_class_uris", collections.BytesKey, col
        ERC721ClassIdsByContractAddr: collections.NewMap(sb, types.ERC721ClassIdsByContractAddrPrefix, "erc721_class_ids_by_contract_ad
        ERC721ContractAddrsByClassId: collections.NewMap(sb, types.ERC721ContractAddrsByClassIdPrefix, "erc721_contract_addrs_by_class_

        EVMBlockHashes: collections.NewMap(sb, types.EVMBlockHashPrefix, "evm_block_hashes", collections.Uint64Key, collections.BytesVa

        queryCosmosWhitelist: queryCosmosWhitelist,
        cosmosCallbackABI:    cosmosCallbackABI,
    }

    // Build schema
    schema, err := sb.Build()
    if err != nil {
        panic(err)
    }

    k.Schema = schema
    k.erc20StoresKeeper = NewERC20StoresKeeper(k)
    k.erc20Keeper, err = NewERC20Keeper(k)
    if err != nil {
```

```go
        panic(err)
    }
    k.erc721Keeper, err = NewERC721Keeper(k)
    if err != nil {
        panic(err)
    }
    k.txUtils = NewTxUtils(k)

    return k
}
```

---

5. EVM 실행 플로우

5.1 Contract Create

실제 구현 (출처: `x/evm/keeper/msg_server.go` lines 33-74)

```go
// Create implements types.MsgServer.
func (ms *msgServerImpl) Create(ctx context.Context, msg *types.MsgCreate) (*types.MsgCreateResponse, error) {
    // 1. Params 가져오기
    params, err := ms.Params.Get(ctx)
    if err != nil {
        return nil, err
    }

    // 2. Publisher 권한 확인
    err = assertAllowedPublishers(ms.ac, params, msg.Sender)
    if err != nil {
        return nil, err
    }

    sender, err := ms.ac.StringToBytes(msg.Sender)
    if err != nil {
        return nil, err
    }

    // 3. Sequence increment 처리 (Cosmos ↔ EVM 차이 조정)
    err = ms.handleSequenceIncremented(ctx, sender, true)
    if err != nil {
        return nil, err
    }

    // 4. Arguments validation
    caller, codeBz, value, accessList, _, err := ms.validateArguments(
        ctx, sender, msg.Code, msg.Value, msg.AccessList, nil, true)
    if err != nil {
        return nil, err
    }

    // 5. EVM contract deployment
    retBz, contractAddr, logs, err := ms.EVMCreate(ctx, caller, codeBz, value, accessList)
    if err != nil {
        return nil, err
    }

    return &types.MsgCreateResponse{
        Result:       hexutil.Encode(retBz),
        ContractAddr: contractAddr.Hex(),
        Logs:         logs,
    }, nil
}
```

5.2 Contract Call

```go
// Call implements types.MsgServer.
func (ms *msgServerImpl) Call(ctx context.Context, msg *types.MsgCall) (*types.MsgCallResponse, error) {
    sender, err := ms.ac.StringToBytes(msg.Sender)
    if err != nil {
        return nil, err
    }

    // Sequence handling
    err = ms.handleSequenceIncremented(ctx, sender, true)
    if err != nil {
        return nil, err
    }

    // Argument validation
    caller, input, value, accessList, contractAddr, err := ms.validateArguments(
        ctx, sender, msg.Input, msg.Value, msg.AccessList, &msg.ContractAddr, false)
    if err != nil {
        return nil, err
    }

    // EVM call execution
    retBz, logs, err := ms.EVMCall(ctx, caller, *contractAddr, input, value, accessList)
    if err != nil {
        return nil, err
    }

    return &types.MsgCallResponse{
        Result: hexutil.Encode(retBz),
        Logs:   logs,
    }, nil
}
```

5.3 Create2 (Deterministic Deployment)

```go
// Create2 implements types.MsgServer.
func (ms *msgServerImpl) Create2(ctx context.Context, msg *types.MsgCreate2) (*types.MsgCreate2Response, error) {
    params, err := ms.Params.Get(ctx)
    if err != nil {
        return nil, err
    }

    // Publisher check
    err = assertAllowedPublishers(ms.ac, params, msg.Sender)
    if err != nil {
        return nil, err
    }

    sender, err := ms.ac.StringToBytes(msg.Sender)
    if err != nil {
        return nil, err
    }

    // Salt validation
    if msg.Salt.IsNegative() {
        return nil, types.ErrInvalidSalt.Wrap("salt is negative")
    }
    salt, overflow := uint256.FromBig(msg.Salt.BigInt())
    if overflow {
        return nil, types.ErrInvalidSalt.Wrap("salt is out of range")
    }

    // ... rest of CREATE2 logic
    retBz, contractAddr, logs, err := ms.EVMCreate2(ctx, caller, codeBz, value, salt, accessList)
    if err != nil {
        return nil, err
    }

    return &types.MsgCreate2Response{
        Result:       hexutil.Encode(retBz),
        ContractAddr: contractAddr.Hex(),
        Logs:         logs,
    }, nil
}
```

---

6. ERC20/ERC721 통합

6.1 Cosmos Coin ↔ ERC20 Bridge

MiniEVM의 **혁신적 기능**: Cosmos native coin을 ERC20으로 자동 래핑

```
Cosmos Bank Module          MiniEVM ERC20 Bridge          EVM Contract
──────────────────          ────────────────────          ────────────
balance[addr][denom]   ↔    ERC20Wrapper contract   ↔   ERC20 token
```

구현:

```go
// ERC20 wrapper address 저장
ERC20WrapperAddr  collections.Item[[]byte]

// Denom ↔ Contract Address 매핑
ERC20DenomsByContractAddr collections.Map[[]byte, string]
ERC20ContractAddrsByDenom collections.Map[string, []byte]

// ERC20 stores (사용자별)
ERC20s       collections.KeySet[[]byte]
ERC20Stores  collections.KeySet[collections.Pair[[]byte, []byte]]
```

6.2 ERC721 Bridge

```
// NFT Class ↔ ERC721 Contract 매핑
ERC721ClassIdsByContractAddr collections.Map[[]byte, string]
ERC721ContractAddrsByClassId collections.Map[string, []byte]
ERC721ClassURIs              collections.Map[[]byte, string]
```

---

7. Cosmos ↔ EVM 브릿지

7.1 EVM → Cosmos 메시지 호출

Precompile 방식이 아닌 **Callback ABI** 사용:

```
// Cosmos callback ABI
cosmosCallbackABI *abi.ABI  // ICosmosCallback interface
```

Solidity 인터페이스:

```
interface ICosmosCallback {
    function executeCosmos(
        string memory moduleRoute,
        bytes memory msgBytes
    ) external returns (bytes memory);
}
```

동작 원리:

1. EVM 컨트랙트가 `ICosmosCallback.executeCosmos()` 호출
2. MiniEVM이 `msgBytes` 를 Cosmos 메시지로 디코딩
3. `msgRouter.Handler(msg)` 실행
4. 결과를 EVM으로 반환

7.2 EVM → Cosmos gRPC 쿼리

```
grpcRouter types.GRPCRouter  // Cosmos gRPC query router

// Query cosmos whitelist (보안)
queryCosmosWhitelist types.QueryCosmosWhitelist
```

사용 예:

```
// Cosmos 잔액 조회
function queryCosmosBalance(address addr, string denom) external view returns (uint256) {
    bytes memory query = abi.encode("/cosmos.bank.v1beta1.Query/Balance", addr, denom);
    bytes memory response = cosmosQuery(query);
    return abi.decode(response, (uint256));
}
```

---

8. JSON-RPC 서버

8.1 지원 네임스페이스

```
jsonrpc/
├── namespaces/
│   ├── eth/          ← eth_* methods
│   ├── net/          ← net_* methods
│   ├── web3/         ← web3_* methods
│   ├── debug/        ← debug_* methods (optional)
│   └── txpool/       ← txpool_* methods (optional)
```

8.2 주요 메소드

카테고리	메소드	구현 여부
Accounts	eth_accounts, eth_sign	✅
Blocks	eth_blockNumber, eth_getBlockByNumber	✅
Transactions	eth_sendRawTransaction, eth_getTransactionReceipt	✅
State	eth_getBalance, eth_getCode, eth_getStorageAt	✅
Gas	eth_estimateGas, eth_gasPrice	✅
Filters	eth_newFilter, eth_getLogs	✅
WebSocket	eth_subscribe (newHeads, logs)	✅

---

9. OPinit 통합

9.1 OPinit이란?

OPinit: Initia의 Optimistic Rollup 프레임워크

```
Initia L1 (Celestia DA)
  ↓ State root submission
MiniEVM L2 (Optimistic execution)
  ↓ Fraud proof window
Finality (7 days)
```

9.2 특징

1. **No Validator Group**: L1 validator 재사용
2. **Built-in Bridge**: L1 ↔ L2 토큰 브릿지
3. **Fraud Proofs**: 7일 챌린지 기간
4. **Fast Finality**: L2 soft finality는 즉시

---

10. Octane vs MiniEVM 비교

10.1 아키텍처 비교

항목	Omni Octane	Initia MiniEVM
EVM 통합 방식	External (Engine API)	Embedded (go-ethereum fork)
Consensus	CometBFT ABCI 2.0	CometBFT ABCI 2.0
L2 Solution	No (L1 chain)	Yes (Optimistic Rollup)
State Storage	EVM client's DB	Cosmos SDK Collections
Block Building	Optimistic (async)	Synchronous
EVM Client	Interchangeable	Fixed (go-ethereum)
Cosmos Integration	Event processors	msgRouter + gRPC
Token Bridge	Event-driven	ERC20/ERC721 wrappers

10.2 기술적 차이

Octane:

- Engine API로 EVM 분리 → 더 모듈화
- Optimistic payload building → 더 빠른 block time
- Event processor → Cosmos 통합

MiniEVM:

- EVM 내장 → 더 단순한 아키텍처
- Collections framework → 통합된 상태 관리
- Callback ABI → 직접적인 Cosmos 호출
- Optimistic Rollup → L2 확장성

10.3 장단점

Octane 장점:

- ✅ EVM client 교체 가능
- ✅ 최신 Ethereum 기능 즉시 지원
- ✅ 더 빠른 block time

Octane 단점:

- ❌ 추가 복잡성 (Engine API)
- ❌ State 분산

MiniEVM 장점:

- ✅ 단순한 아키텍처
- ✅ 통합된 상태 관리
- ✅ Optimistic Rollup L2
- ✅ 직접적인 Cosmos 통합 (msgRouter)

MiniEVM 단점:

- ❌ go-ethereum에 고정
- ❌ 상대적으로 느린 block building

참고 자료

공식 문서

- Initia Documentation
- MiniEVM GitHub
- OPinit Stack

소스 코드

- x/evm module
- JSON-RPC
- Indexer

분석 기준

- **코드 기반**: 실제 `minievm/` 레포지토리 분석
- **버전**: 2024년 9월 기준 최신 코드
- **검증**: 모든 구조체 및 함수 실제 확인

다음 단계

다음 문서에서는 **Berachain Polaris**를 분석합니다:

- Polaris 아키텍처
- Stateful precompiles
- Octane vs MiniEVM vs Polaris 비교

→ 03-berachain-polaris-analysis.md